Classification Methods

Amit Shmidov | Tutorial 2

This presentation largely rely on DL4CV@Weizmann 23’

Supervised Learning

Regression

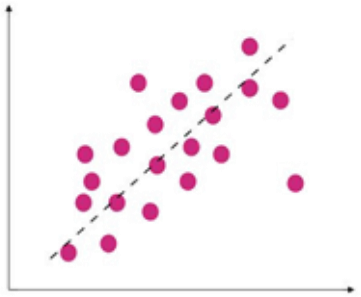
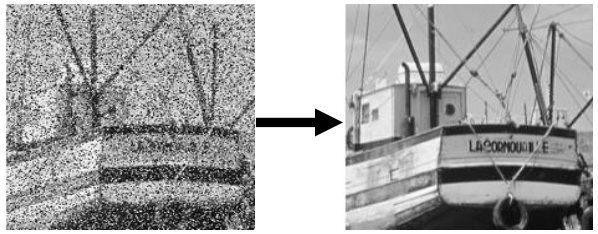


Image denoising



Object localization



Classification

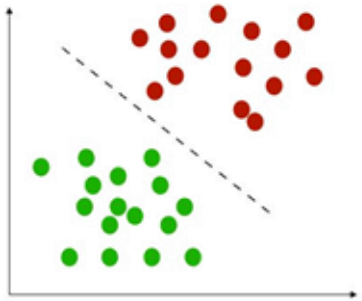
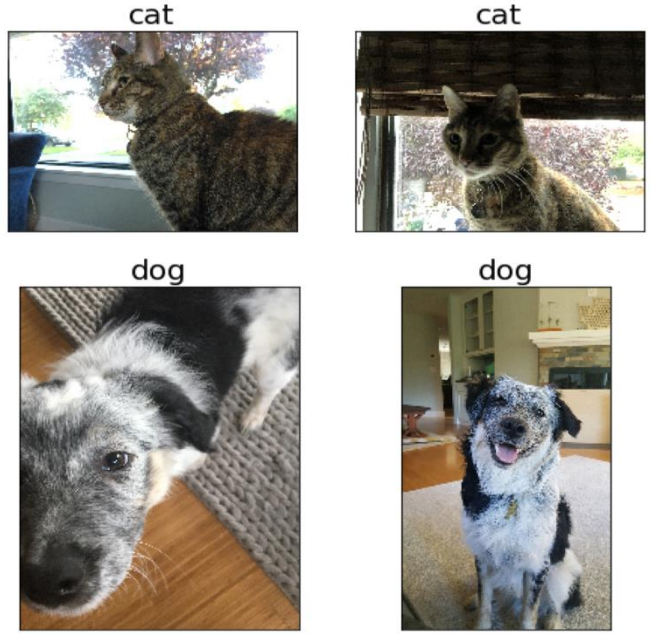


Image classification



Binary Classification



Binary Classification

Samples: $x_i \in R^d$ (e.g., email features)

Labels: $y_i \in \{0,1\}$ (e.g., spam/no spam)

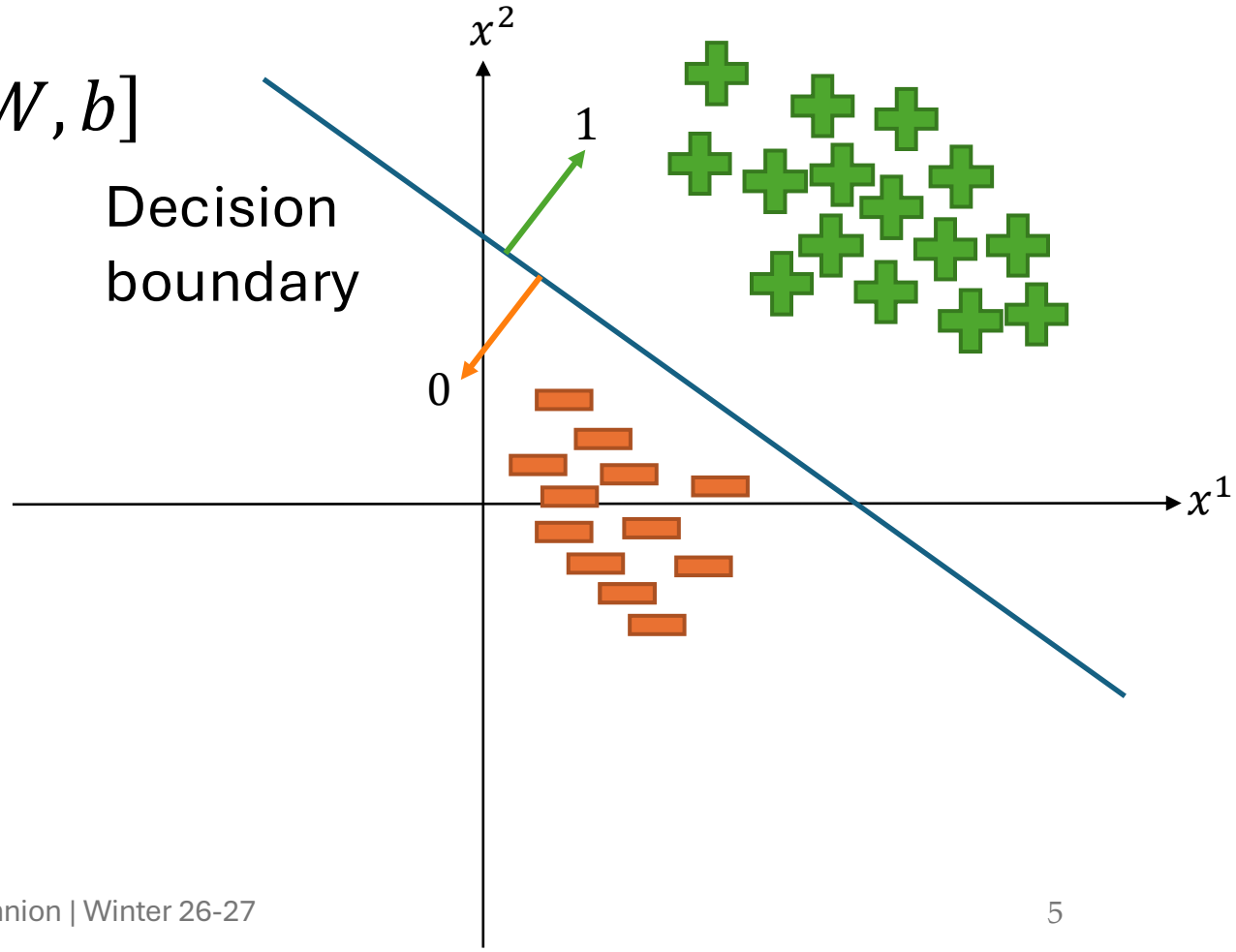
Training set: $\{x_i, y_i\}_{i=1}^m$ (e.g., labeled spam/no spam email features)

Hypothesis: $h_\theta: R^d \to \{0,1\}$ parametrized by θ

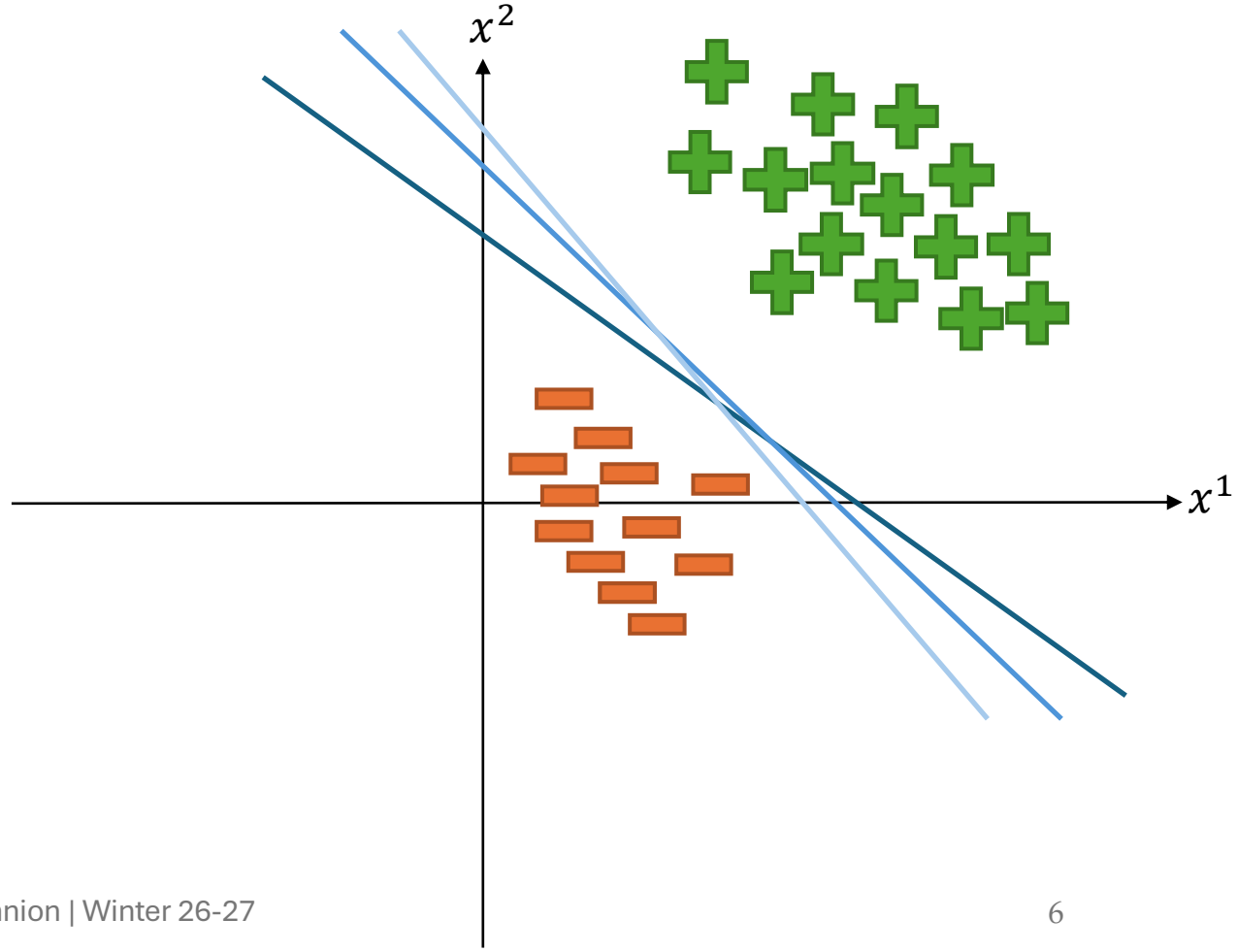
Linear Classification

A linear hypothesis:

$$h_\theta(x_i) = \mathbb{I}(W \cdot x_i + b > 0), \theta = [W, b]$$



Linear Classification



Linear Classification

How to find good W, b?

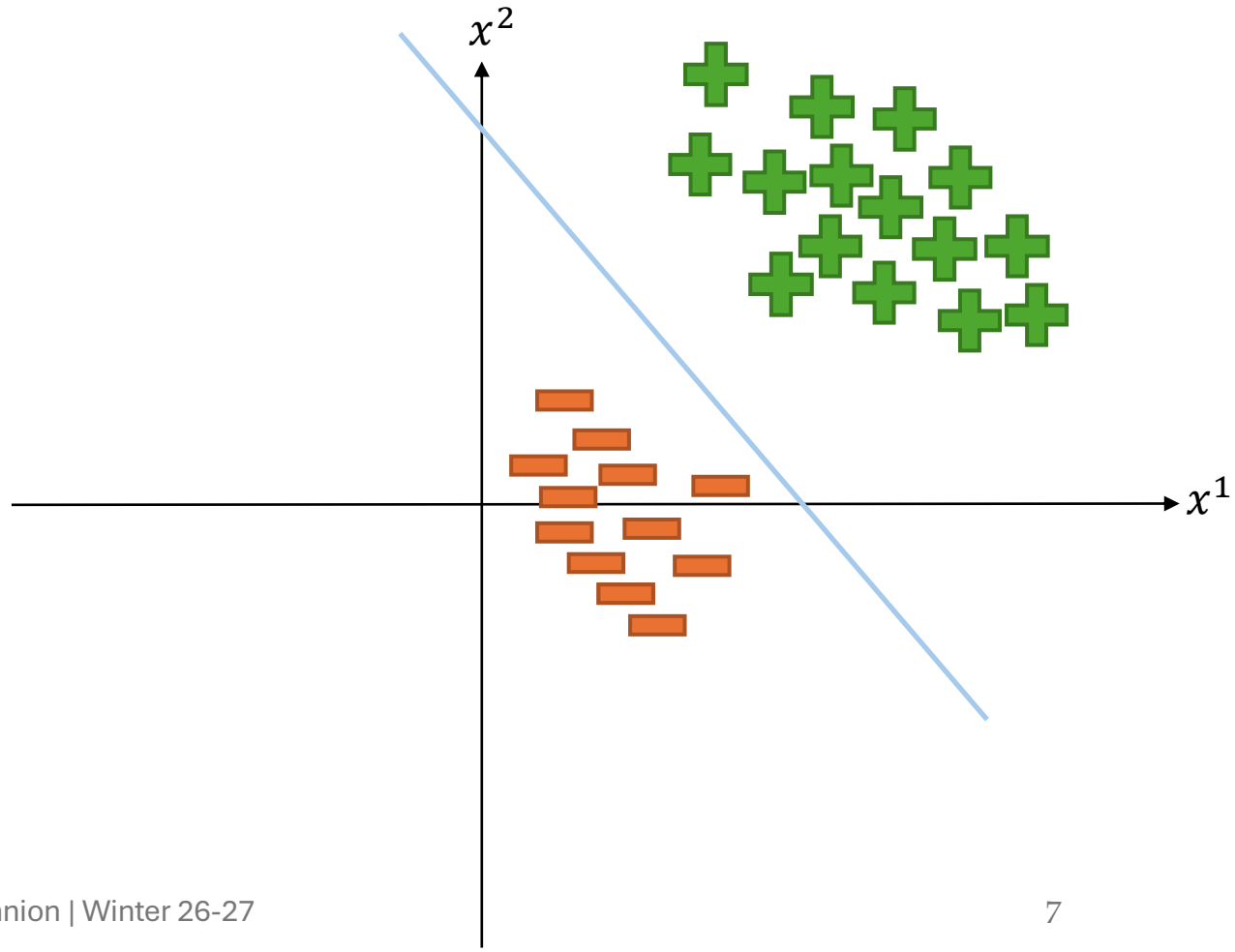
How to interpret the results?

$Wx + b = 1000$ $\quad$ $Wx + b = 0.6$

$\Rightarrow h_{w,b}(x) = 1$ $\quad$ $\Rightarrow h_{w,b}(x) = 1$

For sure! $\quad$ Possibly…

Only intuitive, not measurable



Logistic Regression (Classification)

1. Interpretable results

2. Linear decision boundary

Logistic Regression (Classification)

1. Interpretable results

2. Linear decision boundary

Sigmoid (Logistic Function)

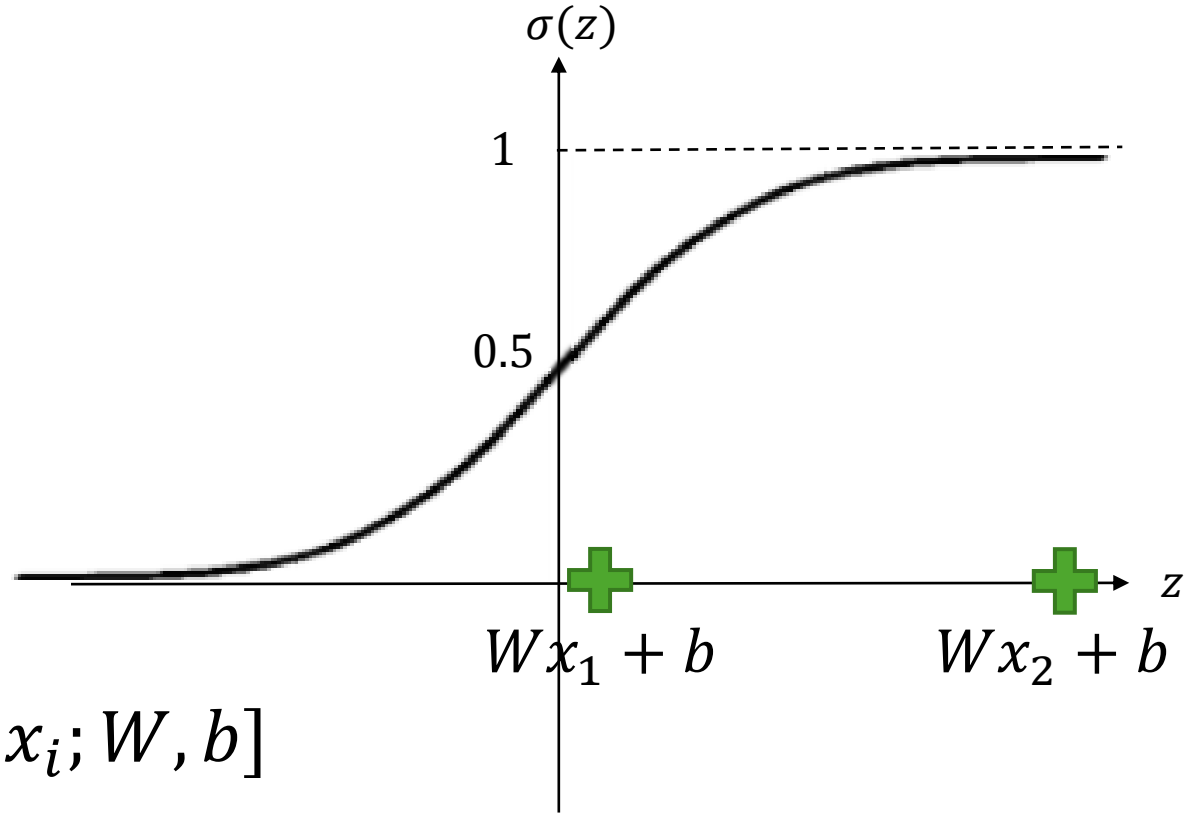
$Wx + b \rightarrow$ Probability

$$z = Wx + b \Rightarrow \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$h_{W,b}(x_i) = \mathbb{I}(\sigma(z_i) > 0.5)$$

Interpretation: $\sigma(z_i) = \Pr[y_i = 1 | x_i; W, b]$

Flipping the label: $1 - \sigma(z) = \Pr[y_i = 0 | x_i; W, b]$



Logistic Regression (Classification)

1. Interpretable results

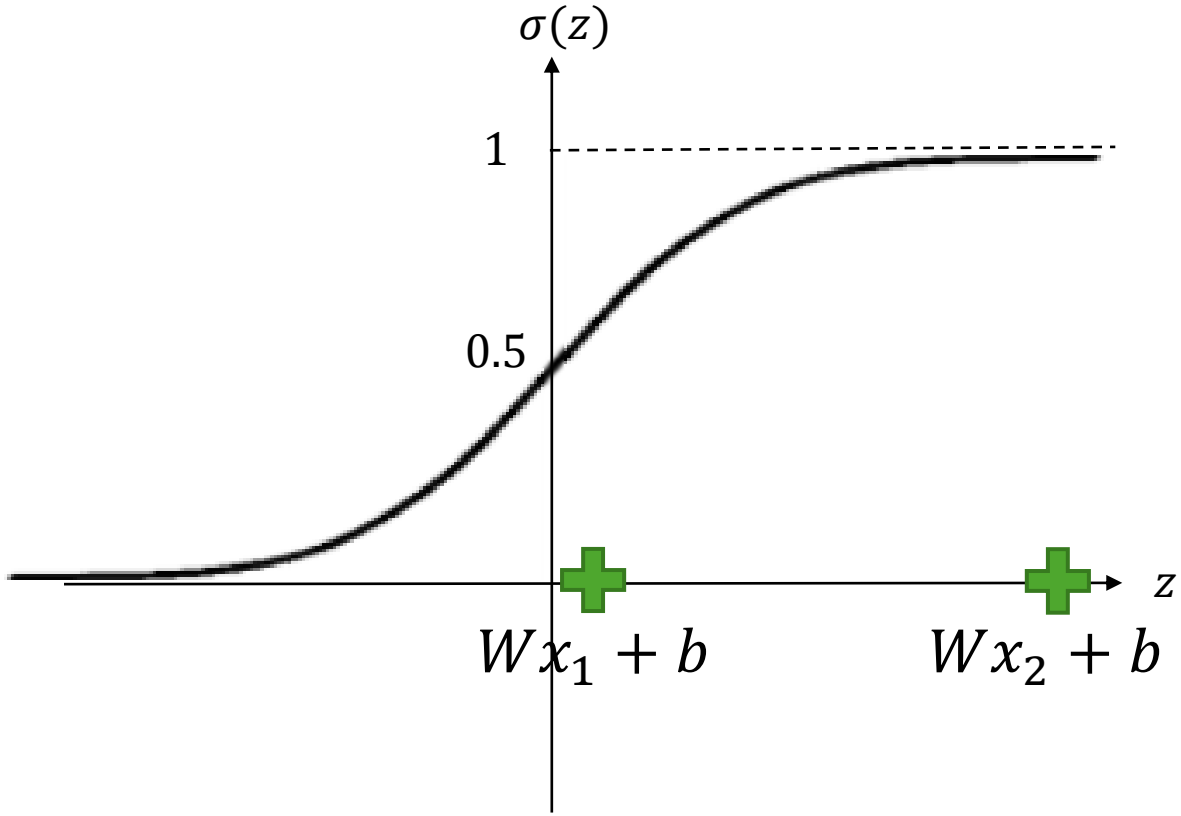
2. Linear decision boundary

Sigmoid (Logistic Function)

$$h_{W,b}(x_i) = \mathbb{I}(\sigma(z_i) > 0.5)$$

$$\sigma(z_i) > 0.5 \Leftrightarrow z > 0$$

⇒ Linear decision boundary



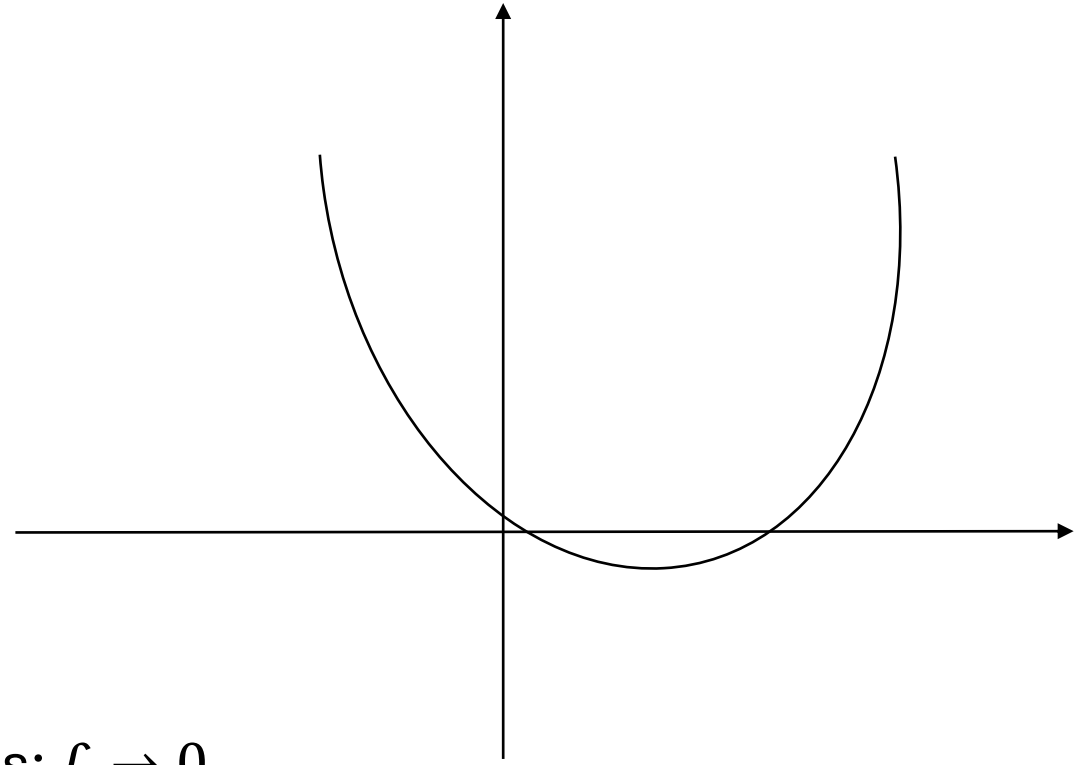
How to find good W, b?

1. Define a convex loss function $\mathcal{L}$
2. Use gradient descent

A Convex Loss Function

A convex function:

1. Every line between every two points is on or above the function.
2. Every local minima of $\mathcal{L}$ is a global minima.



A loss function:

1. $\mathcal{L}$ penalizes errors
2. Wrong predictions: $\mathcal{L} \to \infty$; right predictions: $\mathcal{L} \to 0$

Cross-Entropy Loss

Prediction: $\hat{y}_i = \sigma(z_i) = \sigma(W x_i + b)$

Label: y_i

$$\mathcal{L}_i = y_i \cdot (-\log \hat{y}_i) + (1 - y_i) \cdot (-\log(1 - \hat{y}_i))$$

	$\boldsymbol{y_i = 0}$	$\boldsymbol{y_i = 1}$
$\hat{\mathbf{y}}_\mathbf{i} \to \mathbf{0}$	$\mathcal{L}_\mathrm{i} \to 0$	$\mathcal{L}_\mathrm{i} \to \infty$
$\hat{\mathbf{y}}_\mathbf{i} \to \mathbf{1}$	$\mathcal{L}_\mathrm{i} \to \infty$	$\mathcal{L}_\mathrm{i} \to 0$

Cross-Entropy Loss | Intuition

Self information ("surprise"): $\log \frac{1}{p(x)} = -\log p(x)$

Entropy (average surprise):

$$H(p) = E_{x\sim p}[-\log p(x)] = -\sum p(x) \log p(x)$$

The average surprise when the true probabilities p are known.

Cross-Entropy Loss | Intuition

Cross-entropy $H(p, q)$:

The average surprise when the true probabilities are unknown.

What if we only have $q(x)$, an approximation of $p(x)$?

$$H(p, q) = E_{x \sim p}[-\log q(x)] = -\sum p(x) \log q(x)$$

Cross-Entropy Loss | Intuition

$$\begin{cases} p(y_i = 1|x_i) = y_i \\ p(y_i = 0|x_i) = 1 - y_i \end{cases}$$

$$\Downarrow$$

$$H(p,q) = -p(y_i = 1|x_i) \cdot \log \hat{y}_i - p(y_i = 0|x_i) \cdot \log(1 - \hat{y}_i)$$

$$= y_i \cdot (-\log \hat{y}_i) + (1 - y_i) \cdot (-\log(1 - \hat{y}_i))$$

$$= \mathcal{L}_i$$

Gradient Descent

$$x_i \in R^{d\times 1}, y_i \in \{0,1\}$$

Prediction

$$\hat{y}_i = \sigma(W x_i + b)\ (W \in R^{1\times d}, b \in R \Rightarrow \hat{y}_i \in [0,1])$$

Calculate Loss

$$\mathcal{L}_i = y_i \cdot (-\log \hat{y}_i) + (1 - y_i) \cdot (-\log(1 - \hat{y}_i))$$

Update weights

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T (\hat{y}_i - y_i)$$

Update bias

$$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - y_i)$$

Stochastic Mini-Batch Gradient Descent

Repeat until convergence:

Sample a mini batch $\{x_i, y_i\}_{i=1}^{m}$:

Calculate $\mathcal{L}_i$ for each pair

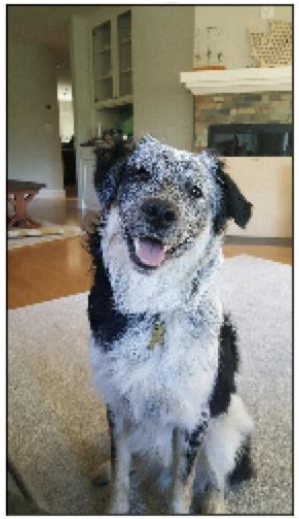
$$\mathcal{L} = \frac{1}{m}\sum_i \mathcal{L}_i$$

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W}$$

$$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}}{\partial b}$$

Multi-class Classification













Multi-class Classification

Samples: $x_i \in R^d$ (e.g., email features)

Labels: $y_i \in \{1, \dots, C\}$ (e.g., spam/no spam/unknown)

Training set: $\{x_i, y_i\}_{i=1}^m$

Hypothesis: $h_\theta : R^d \to [C]$ parametrized by θ

Softmax Classifier

Generalizes logistic regression to multi-class.

Begin with a linear transformation: $z = Wx + b$

Logistic regression	**Softmax classifier**
$W \in R^{1\times d}, b \in R$	$W \in R^{C\times d}, b \in R\text{^}C$

Apply a non-linearity to transform z to probability $\hat{y}$:

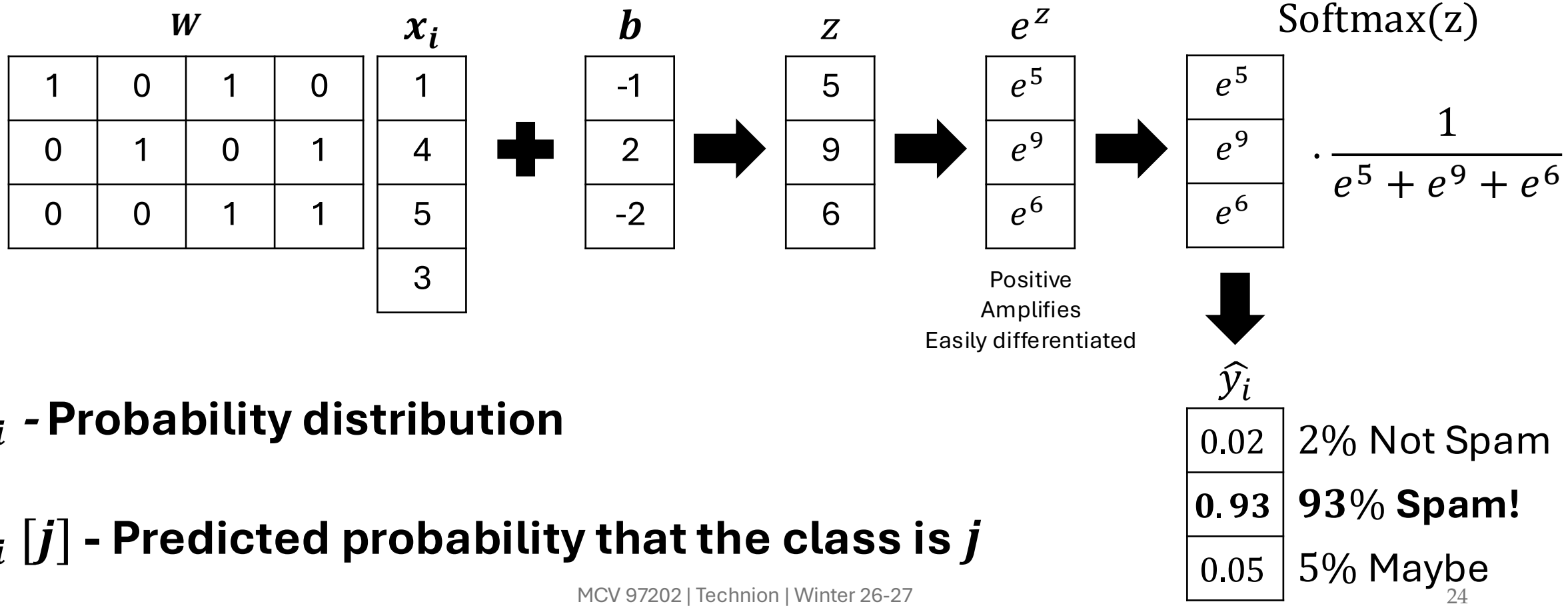
Sigmoid	Softmax

Pick the event with the highest probability:

$\mathbb{I}(\hat{y} > 0.5)$	$\text{argmax}(\hat{y})$

Softmax Function

Given a text x_i: not spam (0), spam (1), unknown (2)



$\hat{y}_i$ - Probability distribution

$\hat{y}_i\ [j]$ - Predicted probability that the class is j

Softmax Function

Converts z to a probability distribution over the classes C:

$$z = Wx + b, \qquad z \in R^C$$

$$f(z)^i = \frac{e^{z^i}}{\sum_{j\in[C]} e^{z^j}}$$

Sigmoid $\frac{1}{1+e^{-z}} = \frac{e^z}{e^z+1}$ is a relaxed version for $C = 2$ and $z^1 = 0$!

$$\hat{y} = [f(z)^0, \dots, f(z)^{C-1}], \sum_{j\in[C]} \hat{y}^j = 1$$

Cross Entropy Loss | Softmax Classifier

$p(y_i = c|x)$ is the GT, $\hat{y}_i^c = q(\hat{y}_i = c|x)$ is the prediction

$p(y_i = c\|x_i)$	
0	Spam
1	**Not spam!**
0	Maybe

$\hat{y}_i$	
0.02	2% Spam
0.93	**93% Not spam!**
0.05	5% Maybe

$$H(p,q) = -\sum_{c\in[C]} p(y_i = c|x) \cdot \log(\hat{y}_i^c) = -\log\left(\hat{y}_i^1\right)$$

Cross Entropy Loss | Softmax Classifier

With C classes and training examples $\{x_i, l_i\}$:

$$\mathcal{L}_i = H(p,q) = -\sum_{c\in[C]} p(y_i = c|x_i)\cdot \log(\hat{y}_i^c) = -\log\left(\hat{y}_i^{l_i}\right)$$

$$= -\log\frac{e^{z_i^{l_i}}}{\sum_{j\in[C]} e^{z_i^j}}$$

Gradient Descent

$$x_i \in R^{d\times 1}, y_i \in \{0,1\}$$

Prediction	$\hat{y}_i = \sigma(Wx_i + b)\ (W \in R^{1\times d}, b \in R \Rightarrow \hat{y}_i \in [0,1])$
Calculate Loss	$\mathcal{L}_i = y_i \cdot (-\log \hat{y}_i) + (1 - y_i) \cdot (-\log(1 - \hat{y}_i))$
Update weights	$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T (\hat{y}_i - y_i)$
Update bias	$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - y_i)$

Gradient Descent | Softmax Classifier

$$x_i \in R^{d\times 1}, y_i \in \{0,1,\dots,C-1\}$$

Prediction	$\hat{y}_i = \sigma(Wx_i + b)\ (W \in R^{1\times d}, b \in R \Rightarrow \hat{y}_i \in [0,1])$
Calculate Loss	$\mathcal{L}_i = y_i \cdot (-\log \hat{y}_i) + (1-y_i)\cdot(-\log(1-\hat{y}_i))$
Update weights	$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T(\hat{y}_i - y_i)$
Update bias	$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - y_i)$

Gradient Descent | Softmax Classifier

$$x_i \in R^{d\times 1}, y_i \in \{0,1,\dots,C-1\}$$

Prediction	$\hat{y}_i = f(Wx_i + b)$ $(W \in R^{C\times d}, b \in R^C \Rightarrow \sum_{j\in[C]} \hat{y}_i^j = 1)$
Calculate Loss	$\mathcal{L}_i = y_i \cdot (-\log \hat{y}_i) + (1-y_i)\cdot(-\log(1-\hat{y}_i))$
Update weights	$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T(\hat{y}_i - y_i)$
Update bias	$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - y_i)$

Gradient Descent | Softmax Classifier

$$x_i \in R^{d\times 1}, y_i \in \{0,1,\dots,C-1\}$$

Prediction

$$\hat{y}_i = f(Wx_i + b)\ (W \in R^{C\times d}, b \in R^C \Rightarrow \textstyle\sum_{j\in[C]} \hat{y}_i^j = 1)$$

Calculate Loss

$$\mathcal{L}_i = -\sum_{c\in[C]} p(y_i = c|x_i) \cdot \log(f(z)^c)$$

Update weights

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T(\hat{y}_i - y_i)$$

Update bias

$$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - y_i)$$

Gradient Descent | Softmax Classifier

$$x_i \in R^{d\times 1}, y_i \in \{0,1,\dots,C-1\}$$

Prediction

$$\hat{y}_i = f(Wx_i + b)\ (W \in R^{C\times d}, b \in R^C \Rightarrow \textstyle\sum_{j\in[C]} \hat{y}_i^j = 1)$$

Calculate Loss

$$\mathcal{L}_i = -\sum_{c\in[C]} p(y_i = c|x_i) \cdot \log(f(z)^c)$$

Update weights

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial W} = W - \alpha \cdot x_i^T(\hat{y}_i - \delta_{y_i})$$

Update bias

$$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}_i}{\partial b} = b - \alpha \cdot (\hat{y}_i - \delta_{y_i})$$

Practical Considerations

- Mini batches

- Numerical stability

Stochastic Mini-Batch Gradient Descent

Repeat until convergence:

Sample a mini batch $\{x_i, y_i\}_{i=1}^{m}$:

Calculate $\mathcal{L}_i$ for each pair

$$\mathcal{L} = \frac{1}{m}\sum_i \mathcal{L}_i$$

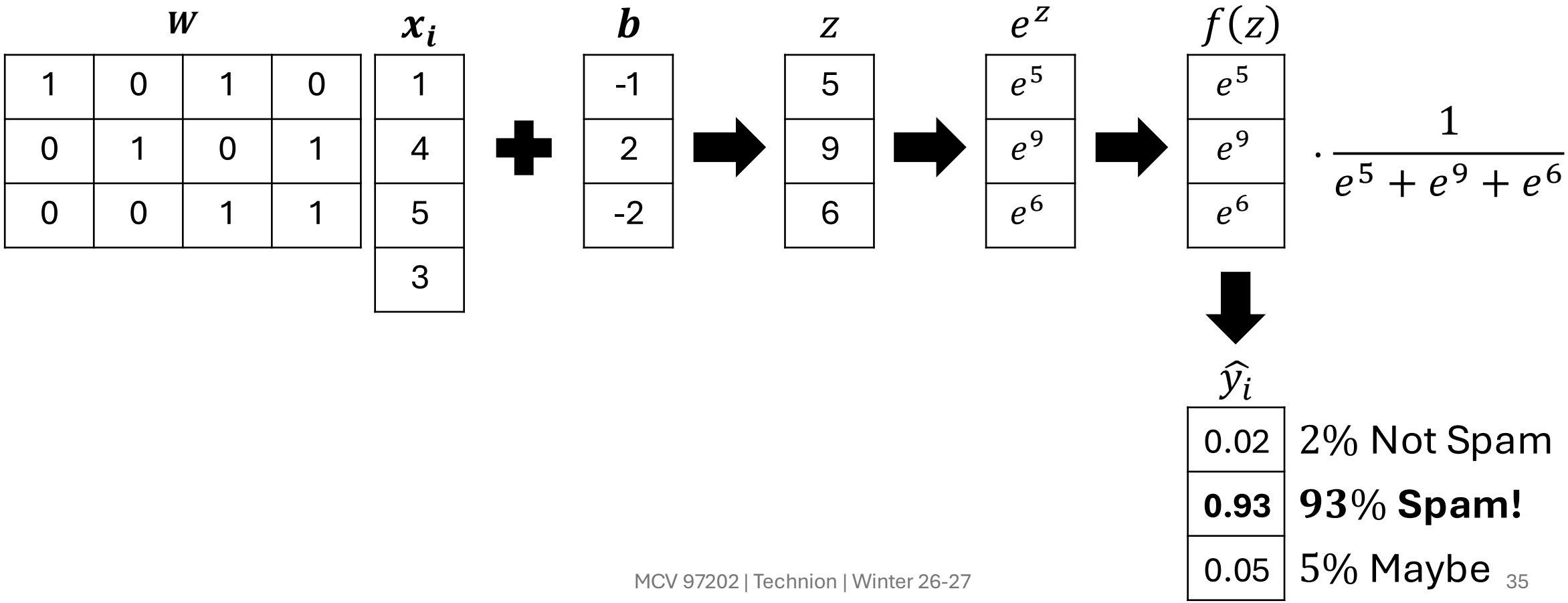
$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W}$$

$$b \leftarrow b - \alpha \cdot \frac{\partial \mathcal{L}}{\partial b}$$

Inefficient sequentially!

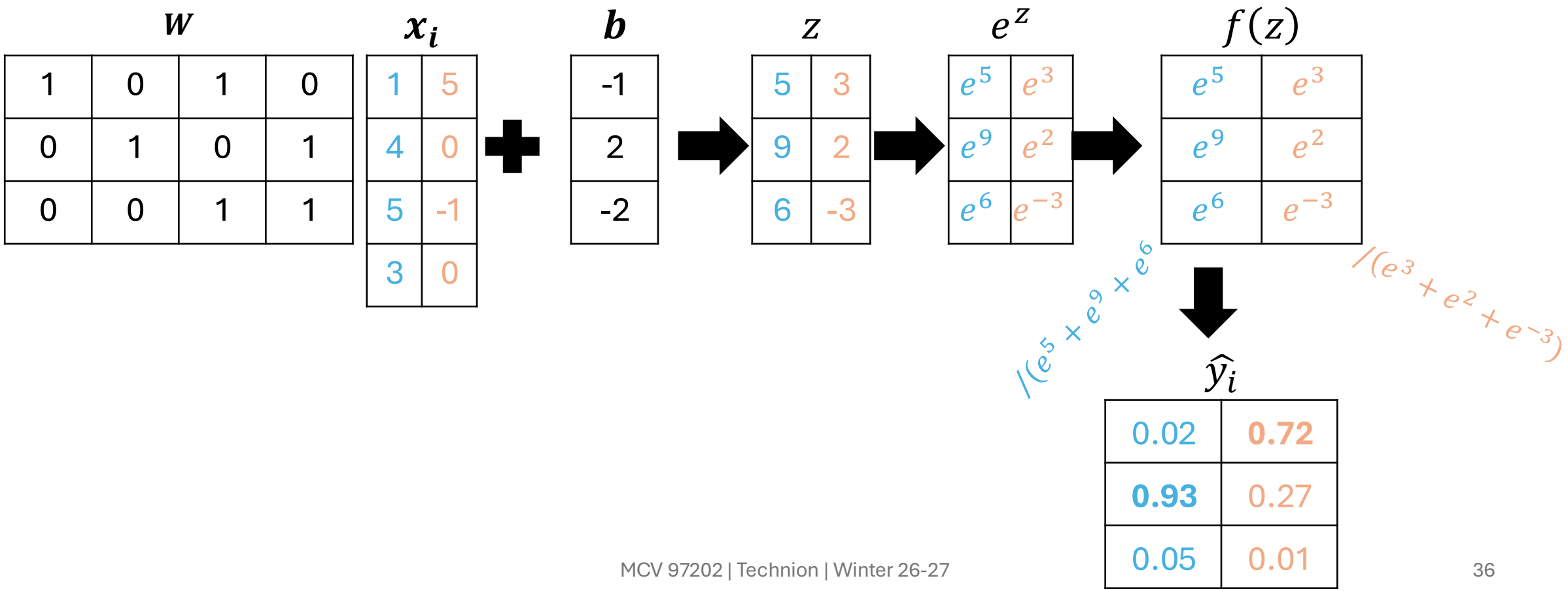
Softmax Function

Given a text x_i: **Not Spam (0), Spam(1)** or **Maybe (2)**



Softmax Function | Mini Batches

Given a text x_i: **Not Spam (0), Spam(1)** or **Maybe (2)**



Mini Batches

$$X = [x_1, \dots, x_m]^T \in R^{m \times d}$$

$$Y \in R^{m \times C}$$

$$W \in R^{C \times d}, b \in R^C \qquad \text{(unchanged)}$$

$$Z = XW^T + b, Z \in R^{m \times C}$$

$$\hat{Y} = f(Z)$$

$$\mathcal{L} = \frac{1}{m} \sum_i \mathcal{L}_i$$

$$W \leftarrow W - \alpha \cdot \frac{\partial \mathcal{L}}{\partial W} = W - \frac{\alpha}{m} \cdot \left(\hat{Y} - Y\right)^T X$$

Numerical Stability

Softmax: e^z can be extremely large

Cross entropy: $\hat{y}$ possibly close to zero $\Rightarrow \log(\hat{y}_i) \to -\infty$